

Deep learning for time-series

Deep learning for time-series

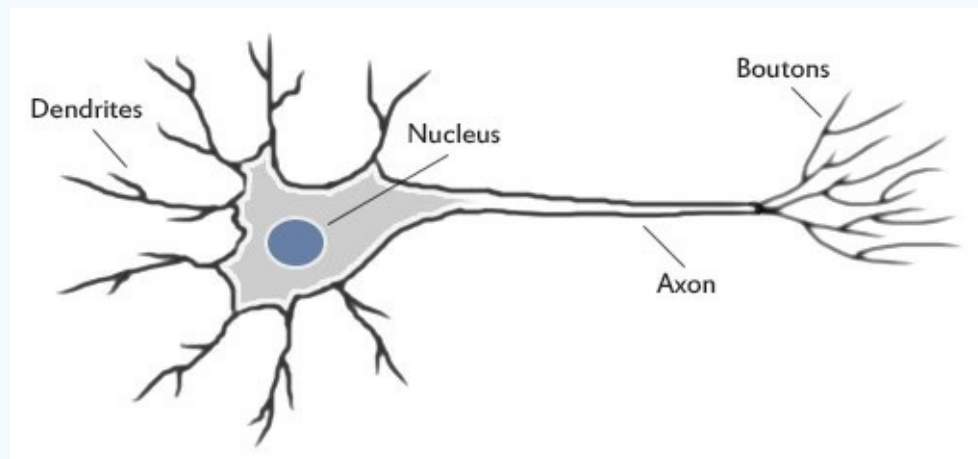
In forecasting, data is often highly erratic, violates the core assumptions of classical techniques, such as Gaussian errors, stationarity, or homoscedasticity.

Deep learning techniques applied to forecasting could overcome many of the challenges faced by classical approaches.

Most importantly, they could provide a way to model non-linear dynamics usually neglected by traditional methods.

Deep learning - Artificial neural network method

- Artificial neural networks are forecasting methods that are based on simple mathematical models of the brain. They allow complex nonlinear relationships between the response variable and its predictors.



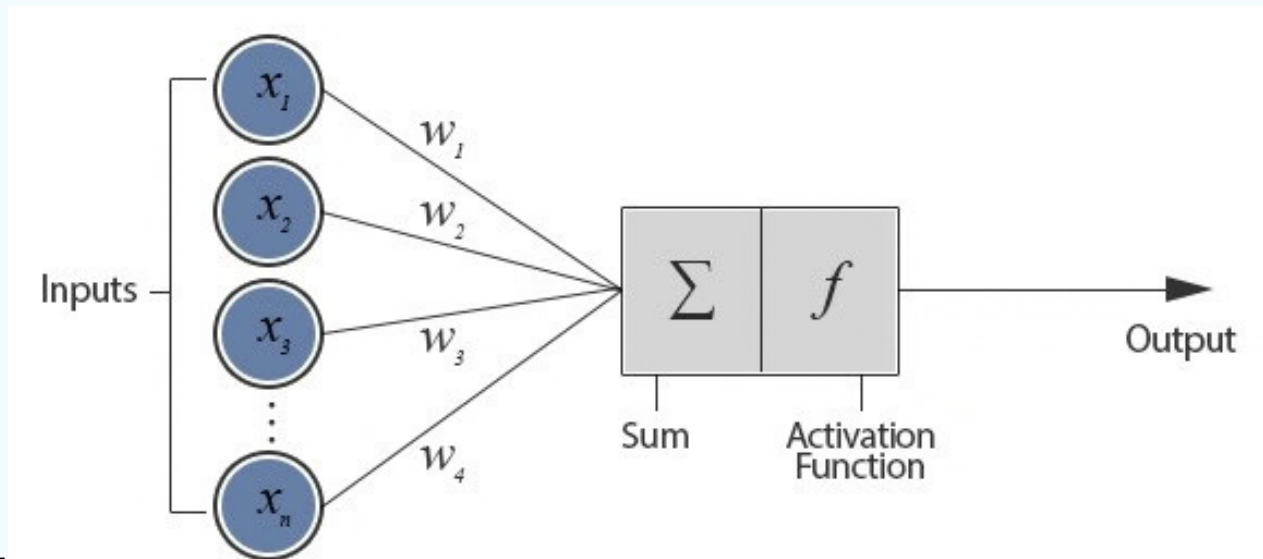
Neural network architecture

- A neural network can be thought of as a network of “neurons” which are organized in layers.
- The predictors (or inputs) form the bottom layer, and the forecasts (or outputs) form the top layer.
- There may also be intermediate layers containing “hidden neurons”.

- Neural networks loosely mimic the way our brains solve the problem: by taking in inputs, processing them and generating an output.
- They *learn* to recognize patterns, but they do this by *training* on labelled datasets.
- Basic of artificial neurons: the **perceptron**, processes inputs and produces an output.

THE PERCEPTRON

- Models of artificial neurons follow the general principles set by the perceptron.



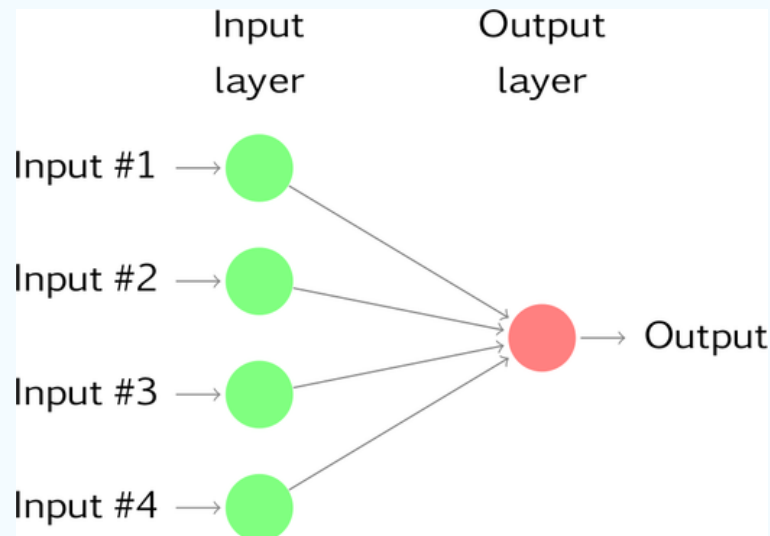
- The network of nodes sends signals in one direction. This is called **a feed-forward network**.
- A neuron connected with n other neurons and thus receives n inputs ($x_1, x_2, \dots, x_n$). This configuration is called a **Perceptron**.

TRAINING IN PERCEPTRONS

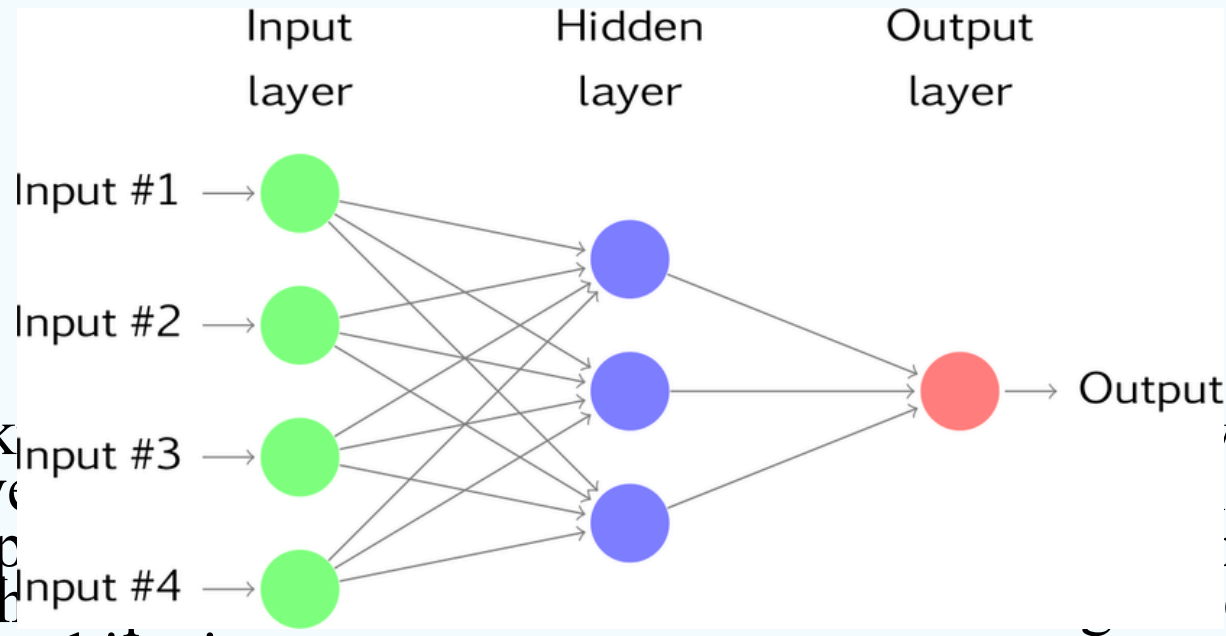
- Input vectors from a training set are presented to the perceptron one after the other and weights are modified according to the following equation,
- For all inputs i ,
$$W(i) = W(i) + a * g'(\text{sum of all inputs}) * (T - A) * P(i),$$

where g' is the derivative of the activation function, and a is the learning rate.
- Here, W is the weight vector, P is the input vector, T is the correct output that the perceptron should have known, and A is the output given by the perceptron.

- The simplest networks contain no hidden layers and are equivalent to linear regressions.
- The coefficients attached to these predictors are called “weights”.
- The forecasts are obtained by a linear combination of the inputs.
- The weights are selected in the neural network framework using a “learning algorithm” that minimizes a “cost function” such as the MSE.



- Once we add an intermediate layer with hidden neurons, the neural network becomes non-linear.



- This is known as a fully connected layer.
- The output layer is a linear combination of the hidden layer outputs.

- The result is then modified by a nonlinear function before being output. For example, the inputs into hidden neuron j in the last figure are combined linearly to give

$$z_j = b_j + \sum_{i=1}^4 w_{i,j} x_i.$$

- In the hidden layer, this is then modified using a nonlinear function such as a sigmoid,

$$s(z) = \frac{1}{1 + e^{-z}}$$

to give the input for the next layer. This tends to reduce the effect of extreme input values, thus making the network somewhat robust to outliers.

The parameters b_1, b_2, b_3 and $w_{1,1}, \dots, w_{4,3}$ are “learned” from the data. The values of the weights are often restricted to prevent them from becoming too large. The parameter that restricts the weights is known as the “decay parameter”, and is often set to be equal to 0.1.

- The weights take random values to begin with, and these are then updated using the observed data.
- Consequently, there is an element of randomness in the predictions produced by a neural network.
- Therefore, the network is usually trained several times using different random starting points, and the results are averaged.
- The number of hidden layers, and the number of nodes in each hidden layer, must be specified in advance.

Neural network autoregression

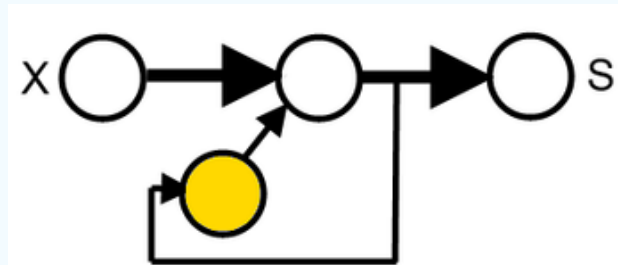
- With time series data, lagged values of the time series can be used as inputs to a neural network, in an autoregression model. This is called neural network autoregression or NNAR model.
- With one hidden layer, the notation NNAR(p,k) indicates there are p lagged inputs and k nodes in the hidden layer.
- For example, NNAR(9,5) model is a neural network with the last nine observations ($y_{t-1}, y_{t-2}, \dots, y_{t-9}$) used as inputs for forecasting the output y_t , and with five neurons in the hidden layer.
- NNAR(p,0) model is equivalent to an ARIMA(p,0,0) model, but without the restrictions on the parameters to ensure stationarity.

Recurrent neural networks and time series

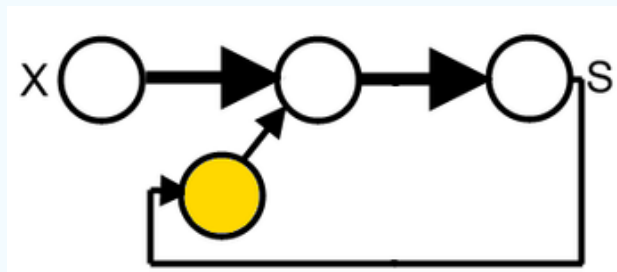
- This is a type of network architecture that implements some kind of memory. This is achieved by implementing some neurons receiving as input the output of one of the hidden layers and injecting their output again in that layer.
- In the **Elman networks**, the inputs of these neurons are taken from the outputs of the neurons in one of the hidden layers, and their outputs are connected back to the inputs of the same layer, providing a memory of the previous state of this layer.

Recurrent neural networks

- The scheme is as in the figure below, where **X** is the input, **S** the output and the yellow node is the neuron in the **context layer**:



- In the **Jordan networks**, the difference is that the input of the neurons in the context layer is taken from the output of the network:



Time Series Prediction with Deep Learning in Keras

Multilayer Perceptron Regression

Step 1: Import all the functions and classes need to use (working SciPy environment with the Keras deep learning library installed).

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

Step 2: Load data set

```
from google.colab import files
import pandas as pd
import io
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
uploaded = files.upload()
```

Chọn tệp TamDaoTemperature_C.csv

TamDaoTemperature_C.csv(text/csv) - 28292 bytes, last modified: 17/10/2025 - 100% done

Saving TamDaoTemperature_C.csv to TamDaoTemperature_C.csv

Step 2: Load data set

To load the dataset as a Pandas dataframe. Then extract from the dataframe the NumPy array in the **column 2** ('Ttb' – Average temperature) and **convert the integer values to floating point values**, which are more suitable for modeling with a neural network.

```
# load the dataset
dataframe =
pd.read_csv('TamDaoTemperature_C.csv',
usecols=[2], engine='python') #
index_col=0, usecols=[1], engine='python')
dataset = dataframe.values
dataset = dataset.astype('float32')
```

Step 2: Load data set

Checking the data set by printing

```
print(dataframe)
```



	Ttb
0	11.5
1	13.0
2	14.4
3	15.2
4	16.1
...	...
1334	22.8
1335	22.9
1336	25.5
1337	21.2
1338	20.6

[1339 rows x 1 columns]

Step 3: Split data into the training and testing parts

With time series data, the sequence of values is important. The code below calculates the index of the split point and separates the data into the training datasets with 80% of the observations used to train your model, leaving the remaining 20% for testing the model.

```
# split into train and test sets
train_size = int(len(dataset) * 0.80)
test_size = len(dataset) - train_size
train, test = dataset[0:train_size,:],
dataset[train_size:len(dataset),:]
print(len(train), len(test))
```



```
1071 268
```

Step 4: Convert array of values into a dataset matrix

Convert the single column of data into a two-column dataset: the first column containing this day's (t) temperature and the second column containing next day's ($t+1$) temperature to be predicted.

Function to create the new dataset takes two arguments: the dataset, which is a NumPy array, and the `look_back`, which is the number of previous time steps to use as input variables to predict the next time period, in this case, defaulted to 1. This default will create a dataset where X is the temperature at a given time (t), and Y is the temperature at the next time ($t + 1$).

Step 4: Convert array of values into a dataset matrix

```
# convert an array of values into a dataset
matrix
def create_dataset(dataset, look_back=1):
    dataX, dataY = [], []
    for i in range(len(dataset)-look_back-1):
        a = dataset[i:(i+look_back), 0]
        dataX.append(a)
        dataY.append(dataset[i + look_back, 0])
    return np.array(dataX), np.array(dataY)
```

Step 5: Prepare the train and test datasets for modeling

```
# reshape into X=t and Y=t+1
look_back = 1
trainX, trainY = create_dataset(train,
look_back)
testX, testY = create_dataset(test,
look_back)
```

Step 6: Create and fit Multilayer Perceptron model

Fit a Multilayer Perceptron model to the training data:

- Use a simple network with **one input**, **one hidden layer** with **eight neurons**, and an output layer.
- The model is fit using **mean squared error**.

```
# create and fit Multilayer Perceptron model
model = Sequential()
model.add(Dense(8, input_shape=(look_back, ),
activation='relu'))
model.add(Dense(1))
model.compile(loss='mean_squared_error',
optimizer='adam')
model.fit(trainX, trainY, epochs=200,
batch_size=2, verbose=2)
```

Step 6: Create and fit Multilayer Perceptron model

```
⇒ Epoch 1/200
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93:
    super().__init__(activity_regularizer=activity_regularizer, **kwargs)
535/535 - 2s - 3ms/step - loss: 100.8228
Epoch 2/200
535/535 - 1s - 2ms/step - loss: 2.8256
Epoch 3/200
535/535 - 1s - 2ms/step - loss: 2.8403
Epoch 4/200
535/535 - 1s - 2ms/step - loss: 2.8438
Epoch 5/200
535/535 - 1s - 2ms/step - loss: 2.8569
...
Epoch 198/200
535/535 - 1s - 1ms/step - loss: 2.8657
Epoch 199/200
535/535 - 1s - 1ms/step - loss: 2.8360
Epoch 200/200
535/535 - 1s - 2ms/step - loss: 2.8825
<keras.src.callbacks.history.History at 0x7e1724179c40>
```

Step 7: Estimate model performance

We can estimate the performance of the model on the train and test datasets.

```
# Estimate model performance
trainScore = model.evaluate(trainX,
trainY, verbose=0)
print('Train Score: %.2f MSE (%.2f RMSE) '
% (trainScore, np.sqrt(trainScore)))
testScore = model.evaluate(testX, testY,
verbose=0)
print('Test Score: %.2f MSE (%.2f RMSE) ' %
(testScore, np.sqrt(testScore)))
```

```
Train Score: 2.81 MSE (1.68 RMSE)
Test Score: 2.76 MSE (1.66 RMSE)
```

Step 8: Generate predictions for training

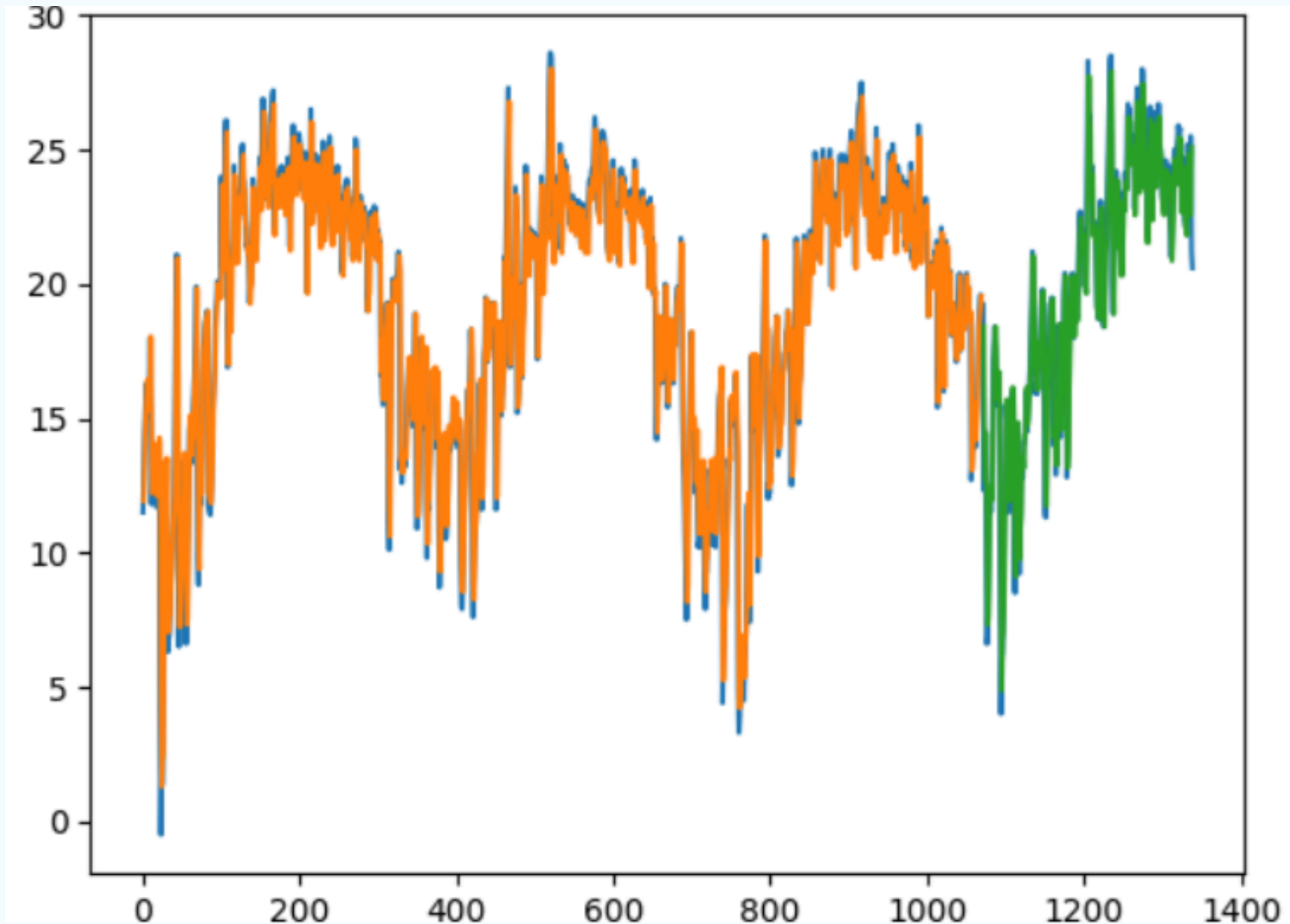
```
# generate predictions for training
trainPredict = model.predict(trainX)
testPredict = model.predict(testX)
# shift train predictions for plotting
trainPredictPlot = np.empty_like(dataset)
trainPredictPlot[:, :] = np.nan
trainPredictPlot[look_back:len(trainPredict)
)+look_back, :] = trainPredict
```

Step 9: Plot baseline and predictions

```
# shift test predictions for plotting
testPredictPlot = np.empty_like(dataset)
testPredictPlot[:, :] = np.nan
testPredictPlot[len(trainPredict)+
    (look_back*2)+1:len(dataset)-1, :] =
testPredict
# plot baseline and predictions
plt.plot(dataset)
plt.plot(trainPredictPlot)
plt.plot(testPredictPlot)
plt.show()
```

the original dataset in blue, the predictions for the training dataset in red, and the predictions on the unseen test dataset in green.

Step 8: Plot baseline and predictions



Time Series Prediction with Deep Learning in Keras

Multilayer Perceptron Using the Window Method

We can phrase the problem so that multiple recent time steps can be used to make the prediction for the next time step.

This is called the window method, and the size of the window is a parameter that can be tuned for each problem.

For example, given the current time (t) to predict the value at the next time in the sequence ($t + 1$), you can use the current time (t) as well as the two prior times ($t-1$ and $t-2$).

When phrased as a regression problem, the input variables are $t-2$, $t-1$, and t , and the output variable is $t+1$.

Step 1': Import functions and classes need to use

Multilayer Perceptron to Predict Average temperature
(t+1, given t, t-1, t-2)

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from pandas import read_csv
```

```
import math
```

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Dense
```

Step 2': Load data set

To load the dataset as a Pandas dataframe. Then extract from the dataframe the NumPy array in the **column 2** ('Ttb' – Average temperature) and **convert the integer values to floating point values**, which are more suitable for modeling with a neural network.

```
# load the dataset
dataframe =
pd.read_csv('TamDaoTemperature_C.csv',
usecols=[2], engine='python
dataset = dataframe.values
dataset = dataset.astype('float32')
```

Step 3': Split data into the training and testing parts

With time series data, the sequence of values is important. The code below calculates the index of the split point and separates the data into the training datasets with 67% of the observations used to train your model, leaving the remaining 33% for testing the model.

```
# split into train and test sets
train_size = int(len(dataset) * 0.67)
test_size = len(dataset) - train_size
train, test = dataset[0:train_size,:],
dataset[train_size:len(dataset),:]
print(len(train), len(test))
```



897 442

Step 4': Convert array of values into a dataset matrix

The **create_dataset()** function allows us to create this formulation of the time series problem by increasing the **look_back** argument from 1 to 3.

We can re-run the example in the previous section with the larger window size. We will increase the network capacity to handle the additional information. The **first hidden layer** is increased to **14 neurons**, and a **second hidden layer** is **added with eight neurons**. The **number of epochs** is also increased to 400.

Step 4': Convert array of values into a dataset matrix

```
# convert an array of values into a
dataset matrix
def create_dataset(dataset, look_back=1):
    dataX, dataY = [], []
    for i in range(len(dataset)-look_back-
1):
        a = dataset[i:(i+look_back), 0]
        dataX.append(a)
        dataY.append(dataset[i + look_back,
0])
    return np.array(dataX), np.array(dataY)
```

Step 5': Prepare the train and test datasets for modeling

```
# reshape dataset
look_back = 3
trainX, trainY = create_dataset(train,
look_back)
testX, testY = create_dataset(test,
look_back)
```

Step 6': Create and fit Multilayer Perceptron model

```
# create and fit Multilayer Perceptron model
model = Sequential()
model.add(Dense(14, input_dim=look_back,
activation='relu'))
model.add(Dense(8, activation='relu'))
model.add(Dense(1))
model.compile(loss='mean_squared_error',
optimizer='adam')
model.fit(trainX, trainY, epochs=400,
batch_size=2, verbose=2)
```

Step 6': Create and fit Multilayer Perceptron model

```
Epoch 1/400
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:105:
    super().__init__(activity_regularizer=activity_regularizer, **kwargs)
447/447 - 2s - 4ms/step - loss: 10.8993
Epoch 2/400
447/447 - 1s - 2ms/step - loss: 4.3935
Epoch 3/400
447/447 - 1s - 2ms/step - loss: 4.1856

. . . .

Epoch 398/400
447/447 - 1s - 1ms/step - loss: 2.7981
Epoch 399/400
447/447 - 1s - 1ms/step - loss: 2.8057
Epoch 400/400
447/447 - 1s - 1ms/step - loss: 2.7941
<keras.src.callbacks.history.History at 0x7c3bbbe65df0>
```

Step 7': Estimate model performance

We can estimate the performance of the model on the train and test datasets.

Estimate model performance

```
trainScore = model.evaluate(trainX, trainY, verbose=0)
print('Train Score: %.2f MSE (%.2f RMSE)' % (trainScore,
np.sqrt(trainScore)))
testScore = model.evaluate(testX, testY, verbose=0)
print('Test Score: %.2f MSE (%.2f RMSE)' % (testScore,
np.sqrt(testScore)))
```

```
Train Score: 2.66 MSE (1.63 RMSE)
```

```
Test Score: 2.27 MSE (1.51 RMSE)
```

Step 8': Generate predictions for training

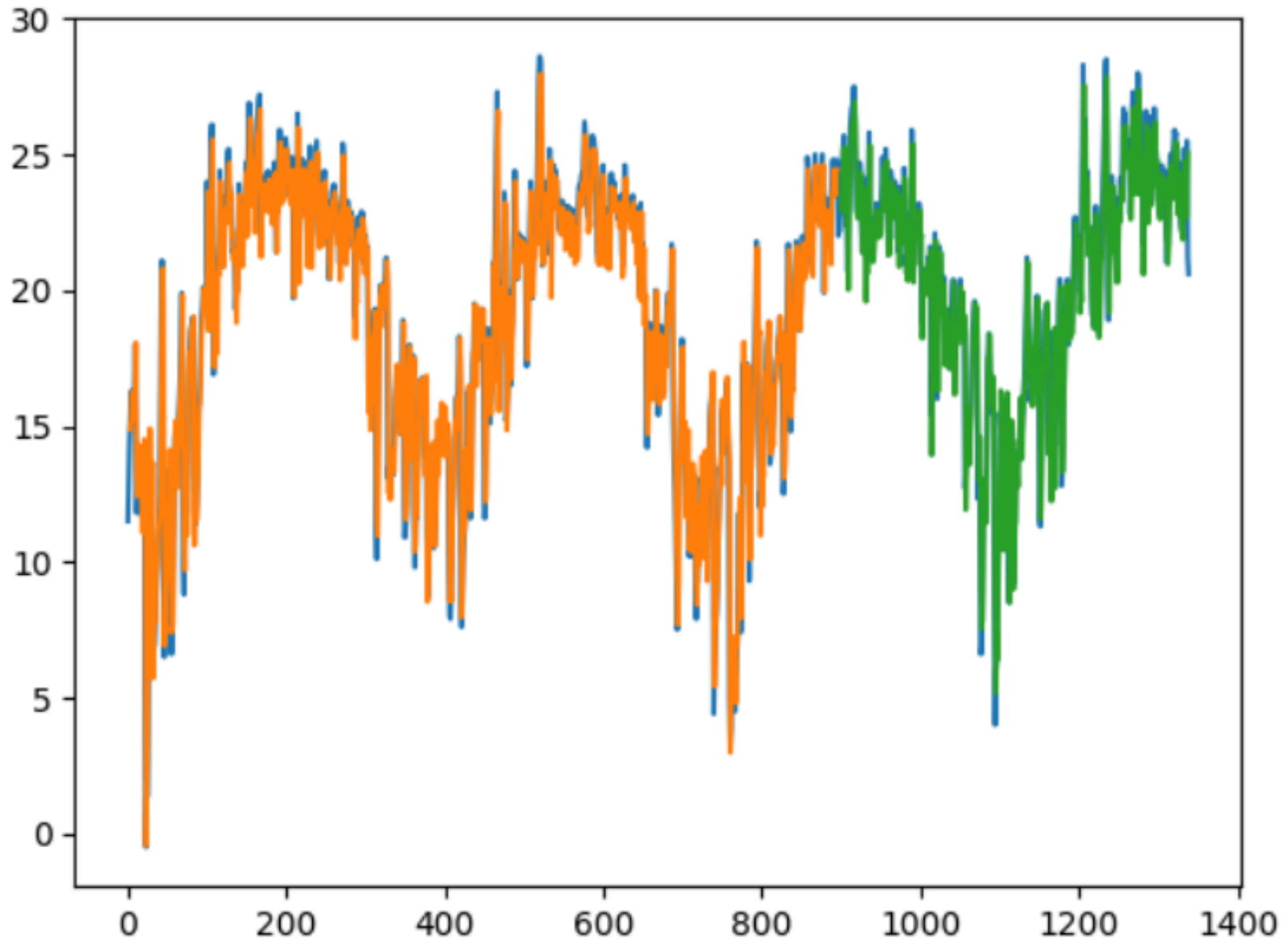
```
# generate predictions for training
trainPredict = model.predict(trainX)
testPredict = model.predict(testX)
# shift train predictions for plotting
trainPredictPlot = np.empty_like(dataset)
trainPredictPlot[:, :] = np.nan
trainPredictPlot[look_back:len(trainPredict)
+look_back, :] = trainPredict
```

Step 9': Plot baseline and predictions

```
# shift test predictions for plotting
testPredictPlot = np.empty_like(dataset)
testPredictPlot[:, :] = np.nan
testPredictPlot[len(trainPredict)+
    (look_back*2)+1:len(dataset)-1, :] =
testPredict
# plot baseline and predictions
plt.plot(dataset)
plt.plot(trainPredictPlot)
plt.plot(testPredictPlot)
plt.show()
```

the original dataset in blue, the predictions for the training dataset in red, and the predictions on the unseen test dataset in green.

Step 9': Plot baseline and predictions



THANK YOU